SIMATS ENGINEERING

ASSESSMENT 3 - EXPERIMENTS

COURSE CODE: CSA1509

COURSE NAME: CLOUD COMPUTING & BIG DATA ANALYTICS

Code of Conduct:

I, **Ashira Bridget Susan S (192524316),** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



CO3-Assessment Tasks

Experiments

Experiment 1: Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.

Experiment 2: Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.

Experiment 3: Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.

Experiment 4: Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.

Experiment 5: Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.

CO3 Experiments: Cloud Storage, REST API, Serverless, Edge/Fog and Deployment Workflow

Cloud Computing Lab Record | Course Outcome 3 (CO3)

Experiment 1: Cloud Storage Comparison (Object vs Block vs File)

Aim

To create and document a comparison of object, block and file storage, and to select the best storage model for backups, VM disks and shared enterprise documents.

Theory

Cloud providers expose three storage models, which differ in how data is organised and accessed.

- **Object storage:** data is stored as flat objects (data + metadata + unique ID/key) in buckets and accessed over HTTP/REST APIs. Examples: Amazon S3, Azure Blob Storage, Google Cloud Storage, OpenStack Swift.
- **Block storage:** data is stored as fixed-size blocks presented to a server as a raw volume (like a hard disk), on which an operating system creates a file system. Examples: Amazon EBS, Azure Managed Disks, Google Persistent Disk, OpenStack Cinder.
- **File storage:** data is stored in a hierarchy of files and folders, shared over network protocols (NFS, SMB/CIFS). Examples: Amazon EFS/FSx, Azure Files, Google Filestore.

Comparison table

Feature	Object storage	Block storage	File storage
Data organisation	Flat namespace of objects with metadata	Raw fixed-size blocks	Hierarchical directories and files
Access method	REST/HTTP API (PUT, GET, DELETE)	Attached to a VM through iSCSI/FC/NVMe; OS formats it	NFS / SMB network mounts
Performance	High throughput, higher latency	Lowest latency, high IOPS	Moderate latency, good for shared access
Scalability	Virtually unlimited (petabytes+)	Limited per volume (TBs)	Scales to large but bounded capacity
Sharing	Shared globally via URLs and policies	Usually one VM at a time	Many clients simultaneously

Feature	Object storage	Block storage	File storage
Modification	Whole-object replace (no in-place edit)	Fine-grained in-place block updates	In-place file edits with locking
Metadata	Rich, custom metadata	None (OS handles it)	Basic (permissions, timestamps)
Durability	Very high (replicated across zones, e.g. 99.999999999%)	High, within a zone, with snapshots	High, depends on service
Cost	Lowest per GB, with tiers (hot, cool, archive)	Highest per GB	Medium
Typical use	Backups, archives, media, data lakes, static websites	Databases, boot volumes, VM disks	Home directories, shared documents, CMS

Procedure (documentation steps)

1. Create an object bucket, upload a file and retrieve it using the console or a CLI/API call.
2. Create a block volume, attach it to a VM, format it (for example ext4) and mount it.
3. Create a file share and mount it on two VMs; confirm that both see the same files.
4. Record observations: access method, speed, sharing behaviour and cost for each.

Example commands (AWS CLI style):

```
# Object storage
aws s3 mb s3://lab-backup-bucket
aws s3 cp backup.zip s3://lab-backup-bucket/

# Block storage
aws ec2 create-volume --size 20 --availability-zone ap-south-1a
sudo mkfs -t ext4 /dev/xvdf && sudo mount /dev/xvdf /data

# File storage (NFS)
sudo mount -t nfs4 fs-1234.efs.ap-south-1.amazonaws.com:/ /shared
```

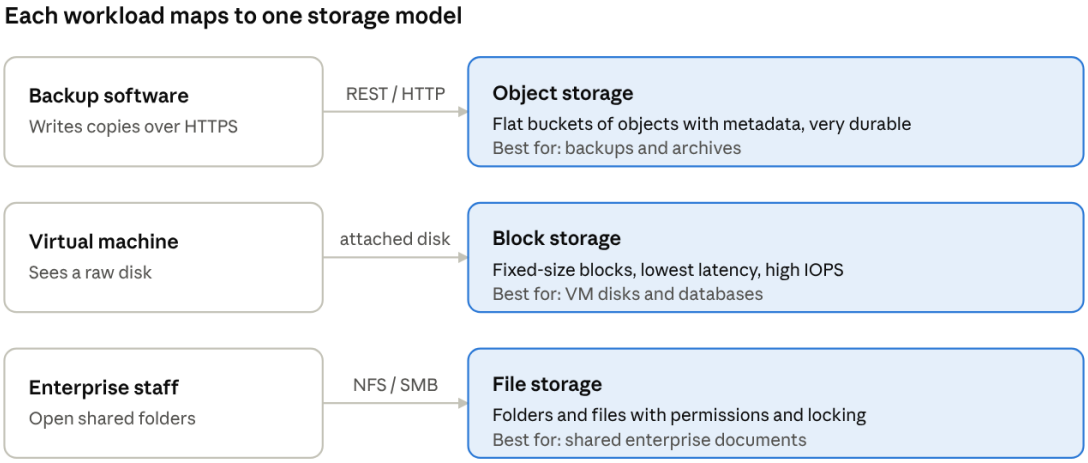
Best storage model for each use case

Use case	Best model	Justification
Backups	**Object storage**	Cheapest per GB, extremely durable, scales without limit, supports versioning, lifecycle policies (move to archive tiers), immutability (WORM) and cross-region replication
VM disks	**Block storage**	Needs low latency, high IOPS and in-place updates at block level; the OS requires a bootable disk-like volume; supports snapshots and resizing

Use case	Best model	Justification
Shared enterprise documents	**File storage**	Many users need simultaneous access to a folder hierarchy with permissions and file locking (SMB/NFS), integrating with Active Directory

Figure

Figure 1 shows how each storage type connects to its consumers.



storage models · 3 workloads

Backups sit in object storage, VM disks on block volumes and shared documents on a file share.

Result / Conclusion

No single storage model fits all needs. Object storage is best for backups, block storage for VM disks, and file storage for shared enterprise documents. In practice, enterprises combine all three (for example, VM disk snapshots are exported to object storage for backup).

Experiment 2: REST API Workflow for a Cloud-Based Student Portal

Aim

To develop a simple REST API workflow for a cloud-based student portal and show the request, response, authentication and deployment flow.

Theory

REST (Representational State Transfer) is a stateless architectural style where resources are identified by URLs and manipulated using HTTP methods (GET, POST, PUT, DELETE), usually exchanging JSON. In a cloud student portal, a front end (web or mobile) calls the API, which authenticates the user, reads or writes the database and returns JSON.

API design

Endpoint	Method	Purpose	Auth
`/api/auth/login`	POST	Authenticate and obtain token	Public
`/api/students/{id}`	GET	View student profile	Student (own) / Admin
`/api/students/{id}/grades`	GET	View grades	Student (own) / Faculty
`/api/courses`	GET	List courses	Any logged-in user
`/api/enrollments`	POST	Enroll in a course	Student
`/api/students/{id}`	PUT	Update profile	Student (own) / Admin
`/api/enrollments/{id}`	DELETE	Drop a course	Student / Admin

Authentication flow (token-based, JWT)

1. The student submits credentials to `/api/auth/login` over HTTPS.
2. The server verifies the credentials against the user database (passwords stored hashed with bcrypt).
3. On success, the server issues a signed **JWT access token** (short expiry, for example 15 minutes) and optionally a refresh token.
4. The client sends the token in the `Authorization: Bearer <token>` header with each later request.
5. An API gateway or middleware validates the signature, expiry and role before the request reaches the business logic.

Sample request and response

1. Login

```
POST /api/auth/login HTTP/1.1
Host: portal.example.edu
```

```
Content-Type: application/json

{ "username": "s2023015", "password": "********" }
```

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "token": "eyJhbGciOiJIUzI1NiIs...", "expiresIn": 900, "role": "student" }
```

2. Get grades (authenticated)

```
GET /api/students/2023015/grades HTTP/1.1
Host: portal.example.edu
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "studentId": 2023015, "grades": [ { "course": "CS301", "grade": "A" }, { "course":
"CS305", "grade": "B+" } ] }
```

3. Enroll in a course

```
POST /api/enrollments HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json

{ "studentId": 2023015, "courseId": "CS310" }
```

```
HTTP/1.1 201 Created
{ "enrollmentId": 8841, "status": "confirmed" }
```

Error responses: `400 Bad Request` (invalid input), `401 Unauthorized` (missing or invalid token), `403 Forbidden` (role not permitted), `404 Not Found`, `500 Internal Server Error`.

Minimal implementation sketch (Node.js / Express)

```
const express = require('express');
const jwt = require('jsonwebtoken');
const app = express();
app.use(express.json());

function auth(req, res, next) {
  const token = (req.headers.authorization || '').split(' ')[1];
  try { req.user = jwt.verify(token, process.env.JWT_SECRET); next(); }
  catch (e) { res.status(401).json({ error: 'Unauthorized' }); }
}

app.post('/api/auth/login', async (req, res) => {
  // verify credentials against the database, then:
  const token = jwt.sign({ id: 2023015, role: 'student' },
                         process.env.JWT_SECRET, { expiresIn: '15m' });
  res.json({ token, expiresIn: 900 });
});
```

```
app.get('/api/students/:id/grades', auth, async (req, res) => {
  // query the database for the grades of req.params.id
  res.json({ studentId: req.params.id, grades: [] });
});

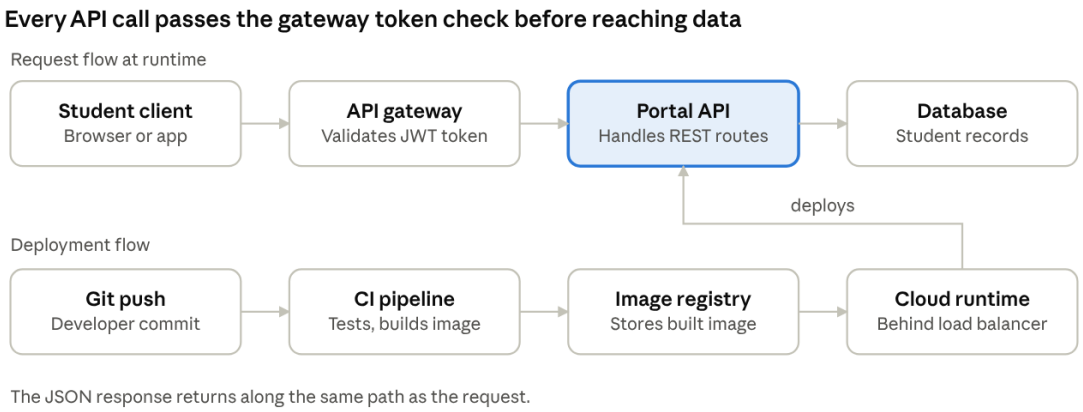
app.listen(3000);
```

Deployment flow

1. Developer pushes code to a Git repository.
2. A CI/CD pipeline (GitHub Actions, Jenkins or CodePipeline) runs tests, builds a container image and pushes it to a registry.
3. The image is deployed to cloud compute (containers on ECS/Kubernetes, App Service, or a serverless function) behind a load balancer.
4. An API gateway provides routing, TLS termination, rate limiting and token validation.
5. The application connects to a managed database (RDS, Azure SQL) in a private subnet; secrets are held in a secrets manager.
6. Monitoring and logging (CloudWatch, Prometheus) track latency, errors and traffic; autoscaling adjusts capacity.

Figure

Figure 2 shows the request, authentication and deployment flow for the portal.



REST request flow and deployment flow · 8 components

The gateway rejects calls without a valid token, and a CI/CD pipeline delivers each new build to the runtime.

Result / Conclusion

The student portal exposes stateless REST endpoints secured by token-based authentication, delivered through an automated deployment pipeline, and can scale horizontally behind a load balancer.

Experiment 3: Serverless Event Flow for Image Upload Processing

Aim

To design a serverless event flow that processes uploaded images using object storage, function-as-a-service (FaaS) and a notification service.

Theory

Serverless computing lets developers run code in response to events without managing servers. The provider handles provisioning, scaling and availability, and billing is per execution. Key services:

- **Object storage** (S3 / Blob Storage): receives uploads and emits events.
- **Function-as-a-Service** (AWS Lambda / Azure Functions / Google Cloud Functions): runs the processing code.
- **Notification service** (SNS / Event Grid / Pub/Sub): delivers messages by email, SMS or to other services.

Event flow

1. A user uploads an image (for example `photo.jpg`) to the **uploads** bucket, directly from a browser using a pre-signed URL.
2. The object-created event triggers the **processing function** automatically.
3. The function reads the image and validates its type and size, then generates thumbnails and compressed versions (and may add a watermark or extract metadata).
4. The function writes the results to a **processed** bucket (a separate bucket avoids infinite trigger loops) and records metadata (name, size, status) in a database such as DynamoDB.
5. The function publishes a message to a **notification topic**.
6. Subscribers (the user by email or SMS, an admin dashboard, another function) receive the notification, for example "Your image photo.jpg was processed."
7. Failures go to a **dead-letter queue** and are retried, and logs go to the monitoring service.

Sample function (Python, AWS Lambda style)

```
import boto3, io
from PIL import Image

s3 = boto3.client('s3')
sns = boto3.client('sns')

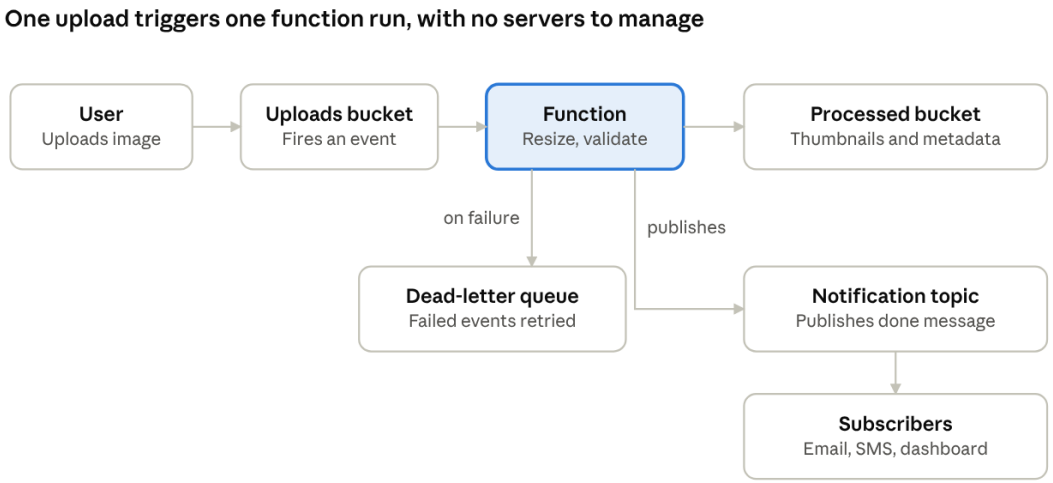
def handler(event, context):
    rec = event['Records'][0]['s3']
    bucket, key = rec['bucket']['name'], rec['object']['key']
    img = Image.open(io.BytesIO(s3.get_object(Bucket=bucket,
Key=key)['Body'].read()))
    img.thumbnail((200, 200))
    buf = io.BytesIO(); img.save(buf, 'JPEG')
    s3.put_object(Bucket='processed-bucket', Key='thumb-' + key, Body=buf.getvalue())
    sns.publish(TopicArn='arn:aws:sns:...:image-done',
                Message='Image ' + key + ' processed successfully')
    return {'status': 'ok'}
```

Design considerations

Aspect	Approach
Security	Least-privilege IAM role for the function, private buckets, pre-signed URLs, file type validation
Scalability	Automatic: each upload triggers its own function instance
Cost	Pay only per invocation and storage; no idle servers
Reliability	Retries, dead-letter queue, idempotent processing
Limits	Function timeout and memory limits; use queues or step functions for heavy workloads
Observability	Logs, metrics and alarms for errors and duration

Figure

Figure 3 shows the event-driven flow from upload to notification.



serverless image processing · 7 components

The upload event starts the function, which writes results, publishes a notification and sends failures to a retry queue.

Result / Conclusion

The serverless design processes images automatically, scales on demand without server management and charges only for actual use, making it ideal for bursty upload workloads.

Experiment 4: Edge Computing vs Fog Computing for Smart Traffic Management

Aim

To compare edge and fog computing for a smart traffic management system, including architecture, latency considerations and deployment location.

Theory

- **Edge computing:** processing happens on or right next to the data source (inside the camera, sensor or roadside unit), with minimal delay.
- **Fog computing:** an intermediate layer of fog nodes (gateways, local servers, micro datacenters) between the edge devices and the cloud; it aggregates and processes data from many edge devices across an area.
- **Cloud:** centralised, highly scalable processing for long-term analytics, model training and city-wide planning.

Smart traffic scenario

Intersections have cameras, vehicle detectors, pedestrian sensors and adaptive traffic signals. The system must detect vehicles and emergency vehicles, adjust signal timing, detect accidents and congestion, and plan city-level routing.

Architecture (three layers)

Layer	Components	Role in traffic system
Edge layer	Smart cameras, inductive loops, radar, roadside units (RSUs), signal controllers with embedded AI	Real-time vehicle detection, immediate signal control, emergency pre-emption (milliseconds)
Fog layer	Neighbourhood or district gateways and local servers at traffic control cabinets or telecom base stations	Coordinates multiple intersections ('green wave'), aggregates data, local congestion prediction, filters data before upload, keeps working if the cloud link fails
Cloud layer	Central datacenter	Long-term storage, big-data analytics, ML model training, city-wide dashboards, reporting and planning

Comparison

Parameter	Edge computing	Fog computing
Location	On the device or at the intersection (camera, RSU, controller)	Intermediate nodes: local servers, gateways, base stations, district cabinets

Parameter	Edge computing	Fog computing
Scope	Single device or intersection	Many devices across an area (a group of intersections)
Latency	Lowest (about 1 to 10 ms)	Low (about 10 to 50 ms), higher than edge but far below cloud (100 ms or more)
Compute power	Limited by device hardware	Greater, with room for aggregation and analytics
Coordination	Isolated decisions	Cross-intersection coordination
Storage	Very small, temporary	Moderate, short-term
Bandwidth use	Greatly reduced (only results are sent)	Reduced through aggregation and filtering
Resilience	Works even if the network drops, but limited view	Local autonomy for a whole zone
Management	Many scattered devices, harder to maintain	Fewer nodes, easier to manage
Traffic examples	Detecting a red-light violation, emergency vehicle pre-emption at one junction	Synchronising signals along a corridor, rerouting during an accident

Latency considerations

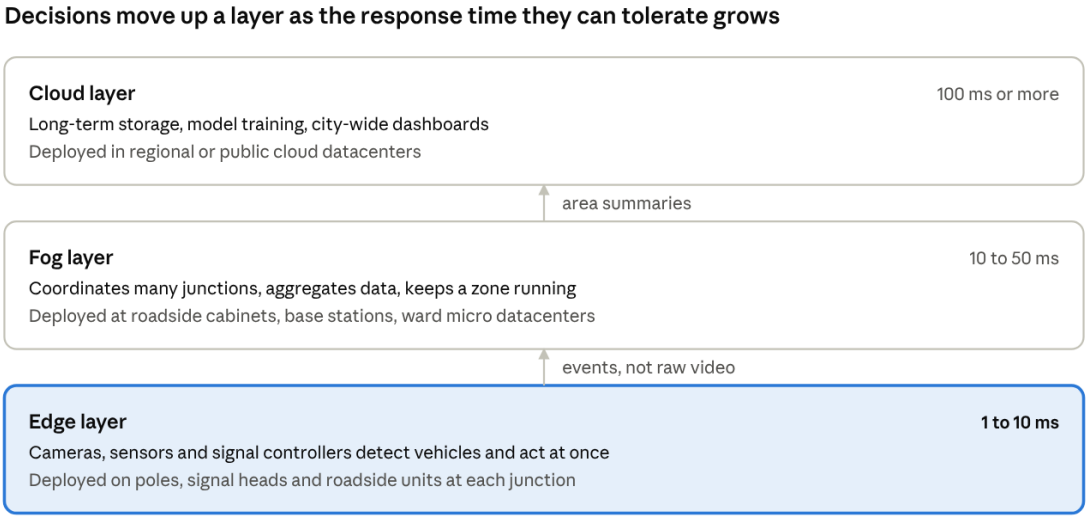
- Safety-critical actions (pre-empting signals for an ambulance, pedestrian detection) require responses within milliseconds, so they must run at the **edge**.
- Area-level optimisation (signal coordination across 10 to 50 junctions) tolerates tens of milliseconds, so it runs in the **fog**.
- Historical analysis and model training tolerate seconds to hours, so they run in the **cloud**.
- Sending raw video to the cloud wastes bandwidth and adds delay, so processing near the source and uploading only events or metadata is preferred.

Deployment location

- **Edge:** poles, traffic signal heads, cameras and roadside units at each intersection.
- **Fog:** roadside traffic cabinets, local police or traffic control rooms, telecom base stations (5G MEC) and ward-level micro datacenters.
- **Cloud:** regional or public cloud datacenters.

Figure

Figure 4 shows the edge, fog and cloud layers of the smart traffic system.



edge, fog and cloud layers · 3 layers

Safety-critical actions stay at the edge, junction coordination runs in the fog, and long-term analytics run in the cloud.

Result / Conclusion

A hybrid design works best: edge for instant, local, safety-critical decisions; fog for coordination and aggregation across intersections; cloud for deep analytics and planning. Fog is preferred where multi-intersection coordination and local resilience matter, edge where minimum latency matters.

Experiment 5: Cloud Application Deployment Workflow

Aim

To demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access and monitoring.

Scenario

A web application (for example a document or photo-sharing app) is deployed in the cloud. A user opens it in a browser, the browser loads the front end, calls a web API, the API reads or writes data in cloud storage and a database, and monitoring tracks health and performance.

Deployment workflow (build side)

1. **Code and commit:** developers push code to Git.
2. **Build and test:** the CI/CD pipeline runs unit tests, builds the front end and packages the back end as a container image.
3. **Provision infrastructure:** infrastructure as code (Terraform / CloudFormation) creates the network, load balancer, compute instances or container service, database and storage buckets.
4. **Deploy:** the image is deployed (rolling or blue-green) to compute instances behind a load balancer; static front-end files go to object storage fronted by a CDN.
5. **Configure:** DNS, TLS certificates, environment variables and secrets are set.
6. **Verify:** health checks and smoke tests confirm the release; roll back if they fail.

Runtime workflow (request side)

Step	Component	Action
1	Client / user	Enters the application URL in the browser
2	DNS and CDN	The domain resolves to the CDN or load balancer; static assets (HTML, CSS, JS) are served from cache or object storage over HTTPS
3	Browser	Renders the page; JavaScript runs and the user interacts (logs in, uploads a file, searches)
4	Web API call	The browser sends an AJAX/fetch request (REST, JSON) with an auth token to the API endpoint
5	Load balancer and API gateway	Terminates TLS, validates the token, applies rate limits and routes to a healthy application instance
6	Application server	Runs business logic and validates the request
7	Storage access	The app reads or writes metadata in the database and files in object storage (via SDK or pre-signed URL)
8	Response	The API returns JSON; the browser updates the page

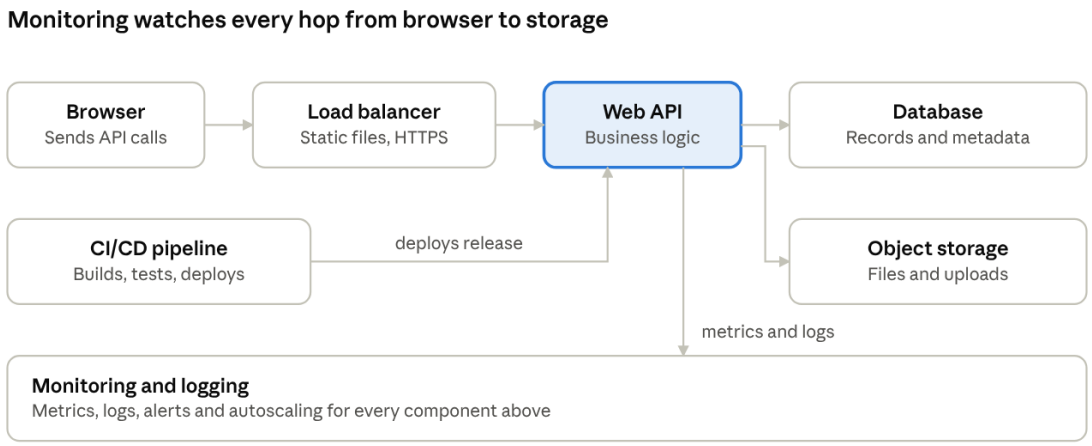
Step	Component	Action
9	Monitoring and logging	Metrics (CPU, latency, error rate), logs and traces are collected; alerts fire on thresholds; autoscaling adds or removes instances

Monitoring and operations

- **Metrics:** CPU and memory, request count, latency, error rate, storage usage.
- **Logs:** application and access logs collected centrally (CloudWatch Logs, ELK).
- **Alerts and dashboards:** notify the team by email or chat when thresholds (for example, 5xx rate above 2%) are exceeded.
- **Autoscaling and self-healing:** unhealthy instances are replaced and capacity follows load.
- **Security and cost:** audit trails, vulnerability scans and budget alarms.

Figure

Figure 5 shows the end-to-end flow from the client to storage and monitoring.



deployment and runtime workflow · 8 components

The browser reaches the Web API through the load balancer, the API uses the database and object storage, and monitoring covers every hop.

Result / Conclusion

The workflow shows how a cloud application moves from code to production and serves a user: client, browser, web API, storage and monitoring work together, with automation providing repeatable deployments, scalability and visibility.

Summary of Experiments

Experiment	Key outcome
1	Backups use object storage, VM disks use block storage, shared documents use file storage
2	Stateless REST API with JWT authentication, deployed through CI/CD behind an API gateway
3	Upload to bucket triggers a function, which writes processed output and sends a notification
4	Edge handles instant local decisions, fog coordinates areas, cloud does deep analytics
5	Client, browser, API, storage and monitoring form the end-to-end deployment and runtime flow